

```
1  Write a C# function that takes a list of integers and returns a new list containing only
2  the even numbers from the original list.
3
4  Provide the code for this function and a brief explanation of how it works.
5
6  using System;
7  using System.Collections.Generic;
8  using System.Linq;
9
10 public class Program
11 {
12     public static void Main()
13     {
14         // Example usage
15         List<int> numbers = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8 };
16         List<int> evenNumbers = GetEvenNumbers(numbers);
17
18         Console.WriteLine("Even numbers: " + string.Join(", ", evenNumbers));
19     }
20
21     public static List<int> GetEvenNumbers(List<int> numbers)
22     {
23         // Use LINQ to filter out even numbers
24         return numbers.Where(n => n % 2 == 0).ToList();
25     }
26 }
27
28 Explanation
29 Function Definition: The GetEvenNumbers function takes a List<int> as an input and
30 returns a new List<int> containing only the
31 even numbers from the input list.
32
33 LINQ Usage: The function uses LINQ (Where clause) to filter out even numbers. The n => n
34 % 2 == 0 lambda expression checks if a number is even.
35
36 Maintaining Order: The Where method preserves the order of the elements in the list, so
37 the output list will maintain
38 the order of even numbers as they appear in the original list.
39
40 Conversion to List: The ToList() method converts the IEnumerable<int> returned by Where
41 into a List<int>.
42
43 1. String Manipulation
44 Question:
45
46 Write a C# function that takes a string and returns the string with all vowels removed.
47
48 Example:
49
50 Input: "Hello World"
51
52 Output: "Hll Wrld"
53
54 Code Example:
55
56 using System;
57
58 public class Program
59 {
60     public static void Main()
61     {
62         // Example usage
63         string input = "Hello World";
64         string result = RemoveVowels(input);
65         Console.WriteLine(result); // Output: Hll Wrld
66     }
67
68     public static string RemoveVowels(string input)
```

```

65     {
66         if (string.IsNullOrEmpty(input))
67             return input;
68
69         // Define vowels
70         char[] vowels = { 'a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U' };
71
72         // Use a StringBuilder to construct the result
73         var result = new System.Text.StringBuilder();
74
75         foreach (char c in input)
76         {
77             // Append character if it's not a vowel
78             if (Array.IndexOf(vowels, c) == -1)
79             {
80                 result.Append(c);
81             }
82         }
83
84         return result.ToString();
85     }
86 }

```

89 Explanation:

91 StringBuilder: Used for efficient string concatenation.

92 Array.IndexOf: Checks if a character is a vowel.

93 Loop: Iterates through each character and appends only non-vowel characters.

## 95 2. Handling Null Values

96 Question:

98 Write a method that takes two strings and returns the concatenation of both strings. If  
99 any of the strings is null, treat it as an empty string.

100 Example:

102 Input: ("Hello", null)

104 Output: "Hello"

106 Code Example:

```

108 csharp
109 Copy code
110 using System;
111
112 public class Program
113 {
114     public static void Main()
115     {
116         // Example usage
117         string result = ConcatenateStrings("Hello", null);
118         Console.WriteLine(result); // Output: Hello
119     }
120
121     public static string ConcatenateStrings(string str1, string str2)
122     {
123         // Use null-coalescing operator to treat null as an empty string
124         return (str1 ?? string.Empty) + (str2 ?? string.Empty);
125     }
126 }

```

127 Explanation:

129 Null-Coalescing Operator (??): Replaces null with an empty string if needed.

130 Concatenation: Combines the strings.

## 132 3. Basic Object-Oriented Programming

133 Question:

134

135 Define a simple class Person with properties FirstName and LastName, and a method GetFullName that returns the full name of the person.

136

137 Code Example:

138

139 csharp

140 Copy code

141 using System;

142

143 public class Program

144 {

145 public static void Main()

146 {

147 // Example usage

148 Person person = new Person { FirstName = "John", LastName = "Doe" };

149 Console.WriteLine(person.GetFullName()); // Output: John Doe

150 }

151 }

152

153 public class Person

154 {

155 public string FirstName { get; set; }

156 public string LastName { get; set; }

157

158 public string GetFullName()

159 {

160 return \$"{FirstName} {LastName}";

161 }

162 }

163 Explanation:

164

165 Properties: Define FirstName and LastName with getters and setters.

166 Method: Concatenates FirstName and LastName to return the full name.

167 4. Basic Exception Handling

168 Question:

169

170 Write a method that divides two integers and returns the result. Handle the case where the divisor is zero by catching the DivideByZeroException and returning 0.

171

172 Example:

173

174 Input: (10, 0)

175

176 Output: 0

177

178 Code Example:

179

180 csharp

181 Copy code

182 using System;

183

184 public class Program

185 {

186 public static void Main()

187 {

188 // Example usage

189 int result = Divide(10, 0);

190 Console.WriteLine(result); // Output: 0

191 }

192

193 public static int Divide(int dividend, int divisor)

194 {

195 try

196 {

197 return dividend / divisor;

198 }

199 catch (DivideByZeroException)

```

200         {
201             return 0; // Handle divide by zero case
202         }
203     }
204 }
205 Explanation:
206
207 Try-Catch: Handles exceptions that may occur during division.
208 Return Value: Returns 0 if an exception occurs.
209 5. Collections
210 Question:
211
212 Write a method that takes a list of integers and returns a dictionary where the keys are
the integers and the values are their squares.
213
214 Example:
215
216 Input: [1, 2, 3]
217
218 Output: {1: 1, 2: 4, 3: 9}
219
220 Code Example:
221
222 csharp
223 Copy code
224 using System;
225 using System.Collections.Generic;
226 using System.Linq;
227
228 public class Program
229 {
230     public static void Main()
231     {
232         // Example usage
233         List<int> numbers = new List<int> { 1, 2, 3 };
234         Dictionary<int, int> result = GetSquares(numbers);
235
236         foreach (var kvp in result)
237         {
238             Console.WriteLine($"{kvp.Key}: {kvp.Value}");
239         }
240     }
241
242     public static Dictionary<int, int> GetSquares(List<int> numbers)
243     {
244         // Use LINQ to create a dictionary from the list
245         return numbers.ToDictionary(n => n, n => n * n);
246     }
247 }
248 Explanation:
249
250 ToDictionary: Creates a dictionary where the key is the integer and the value is its
square.
251 LINQ: Utilized to project and construct the dictionary.
252
253
254 Certainly! Here are concise answers to the frequently asked questions for someone with a
year of experience in C# or ASP.NET:
255
256 ### C# Questions:
257
258 1. **What is the difference between `ref` and `out` parameters in C#?**
259 - **`ref`**: The parameter must be initialized before it is passed to the method.
260 The method can modify the value and the changes will be reflected outside the method.
261 - **`out`**: The parameter does not need to be initialized before being passed to the
method.
262 The method must assign a value to it before returning. It's typically used when the
method needs to return multiple values.
263

```

```

264 2. **Explain the concept of garbage collection in C#. How does it work?**
265     - Garbage collection (GC) in C# is an automatic memory management feature that
        periodically frees
266 up memory used by objects that are no longer referenced. It works by identifying objects
        that are no longer reachable and reclaiming
267 their memory to be reused. The GC runs on its own schedule, but developers can influence
        it using GC.Collect(), though this is rarely necessary.
268
269 3. **What are the key differences between interface and abstract class?**
270     - **Abstract Class**: Can provide some method implementations and state (fields). It
        can have constructors and destructors.
271     It can be partially implemented.
272     - **Interface**: Cannot provide any implementation, only method signatures (in older
        versions of C#) or default
273 implementations (from C# 8.0 onwards). Interfaces support multiple inheritance,
        whereas a class can inherit from only one abstract class.
274
275 4. **Can you explain the async and await keywords and how they work in C#?*
276     - async is used to define a method that can perform asynchronous operations.
        await is used inside async methods to pause
277 execution until the awaited task completes. This allows for non-blocking operations
        and better responsiveness.
278
279 5. **What is LINQ, and can you give an example of how you might use it?*
280     - LINQ (Language Integrated Query) allows querying of collections using a syntax
        similar to SQL. Example:
281         ```csharp
282         var numbers = new List<int> { 1, 2, 3, 4, 5 };
283         var evenNumbers = numbers.Where(n => n % 2 == 0).ToList();
284         ```
285
286 6. **What are delegates and events? How do they differ?*
287     - **Delegates**: Type-safe function pointers that can reference methods with a
        specific signature. They allow methods to be passed as parameters.
288     - **Events**: Encapsulate delegates to provide a publish-subscribe model. They
        restrict direct invocation of the
289 delegate to the class that defines the event, allowing only subscribers to handle the
        event.
290
291 7. **How does exception handling work in C#? Can you explain the try-catch-finally
        block?*
292     - **try**: Contains code that might throw an exception.
293     - **catch**: Catches and handles exceptions thrown by the try block.
294     - **finally**: Contains code that is always executed after the try and catch
        blocks, regardless of whether an exception was thrown or not.
295     It is often used for cleanup code.
296
297 8. **What is the purpose of the using statement in C#?*
298     - The using statement ensures that objects implementing IDisposable are properly
        disposed of when they are no longer needed.
299     It automatically calls the Dispose method on the object, which is crucial for
        releasing unmanaged resources.
300
301 9. **Can you describe what happens during the boxing and unboxing process?*
302     - **Boxing**: Converting a value type (e.g., int) to an object or another
        reference type.
303     This involves creating a new object on the heap and copying the value into it.
304     - **Unboxing**: Converting the object back to the original value type. It requires an
        explicit cast and extracts the value from the object.
305
306 10. **What are tuples in C# and how are they used?*
307     - Tuples are data structures that hold a fixed-size sequence of elements of
        different types.
308     They are useful for returning multiple values from a method without creating a
        separate class. Example:
309         ```csharp
310         var tuple = (Name: "Alice", Age: 30);
311         Console.WriteLine(tuple.Name); // Output: Alice
312         ```

```

```

313
314 ### ASP.NET Questions:
315
316 1. What are the differences between ASP.NET Web Forms, MVC, and Web API?
317 - Web Forms: Uses a drag-and-drop model with server controls and postbacks. Ideal
    for rapid application development but
318 less flexible for complex UI.
319 - MVC (Model-View-Controller): Separates application concerns into Models, Views,
    and Controllers.
320 Provides more control over HTML and is better for complex applications.
321 - Web API: Designed for building RESTful services that can be consumed by various
    clients (web, mobile).
322 Focuses on HTTP-based communication and is used for creating APIs.
323
324 2. Can you explain the ASP.NET lifecycle, from request to response?
325 - The lifecycle includes:
326 1. Initialization: Create an instance of the page.
327 2. Load: Load the page and its controls.
328 3. Postback Event Handling: Handle any postback events.
329 4. Rendering: Generate HTML to send to the client.
330 5. Unload: Cleanup after the page has been fully rendered.
331
332 3. What is dependency injection, and how is it implemented in ASP.NET Core?
333 - Dependency Injection (DI) is a design pattern used to achieve Inversion of Control
    (IoC) by injecting
334 dependencies into a class rather than the class creating them. In ASP.NET Core, DI is
    implemented using the built-in
335 `IServiceCollection` and `IServiceProvider`. You register services in
    `Startup.ConfigureServices` and inject them into constructors.
336
337 4. How does routing work in ASP.NET Core?
338 - Routing maps incoming requests to the appropriate controller and action method. It
    is configured in `Startup.
339 Configure` using the `UseRouting` and `UseEndpoints` methods. Routes can be defined
    using attribute routing or conventional routing.
340
341 5. What are middleware components in ASP.NET Core, and can you provide an example?
342 - Middleware components are pieces of code that handle requests and responses in the
    pipeline. They are added in `Startup.Configure`
343 using `app.UseMiddleware`. Example:
344 ```csharp
345 public void Configure(IApplicationBuilder app)
346 {
347     app.Use(async (context, next) =>
348     {
349         // Custom middleware logic
350         await next.Invoke();
351     });
352     app.UseRouting();
353     app.UseEndpoints(endpoints => { endpoints.MapControllers(); });
354 }
355 ```
356
357 6. How do you handle authentication and authorization in ASP.NET Core?
358 - Authentication: Validate the identity of users (e.g., using cookies, JWT).
    Configured in `Startup.ConfigureServices` with services like `AddAuthentication`.
359 - Authorization: Determine if an authenticated user has permission to access a
    resource. Implemented using policies and roles, configured with `AddAuthorization`.
360
361 7. What is Razor, and how does it differ from Web Forms?
362 - Razor is a view engine in ASP.NET MVC and ASP.NET Core that allows embedding C#
    code within HTML. It provides a more streamlined and readable syntax compared to Web
    Forms, which uses server-side controls and a view state.
363
364 8. Can you explain the concept of model binding in ASP.NET MVC?
365 - Model binding maps HTTP request data (e.g., form fields, query strings) to action
    method parameters or model properties. It automatically converts data from request
    formats into .NET types and populates model objects.
366

```

```

367 9. **How do you implement validation in an ASP.NET Core application?**
368 - Validation is implemented using data annotations (e.g., `[Required]`,
    `[StringLength]`) in model classes. ASP.NET Core also supports custom validation
    through `IValidatableObject` and validation attributes.
369
370 10. **What is Entity Framework Core, and how does it compare to Entity Framework 6?**
371 - Entity Framework Core (EF Core) is a modern, lightweight, and cross-platform ORM
    for .NET. It has a more modular design and improved performance compared to Entity
    Framework 6 (EF6), which is more suited to .NET Framework and has a more mature
    feature set.
372
373 ### General Questions:
374
375 1. **Describe a project or task you worked on using C# or ASP.NET. What were the
    challenges, and how did you overcome them?**
376 - Provide a brief summary of a project or task, highlighting your role, the
    technologies used, challenges faced (e.g., performance issues, integration
    difficulties), and solutions or strategies you implemented to overcome those
    challenges.
377
378 2. **How do you approach debugging in C# or ASP.NET applications?**
379 - Use debugging tools like Visual Studio's debugger to set breakpoints, step through
    code, and inspect variables. Utilize logging to trace execution flow and capture
    error details.
380
381 3. **What tools or practices do you use for version control and collaboration in your
    projects?**
382 - Common tools include Git and platforms like GitHub, GitLab, or Bitbucket. Practices
    include branching strategies (e.g., feature branches), code reviews, and pull
    requests.
383
384 4. **How do you ensure your code is maintainable and scalable?**
385 - Follow coding standards, use design patterns, write unit tests, and ensure code is
    modular and well-documented. Apply principles such as SOLID to promote
    maintainability and scalability.
386
387 5. **Can you explain how you stay updated with new features and best practices in C# or
    ASP.NET?**
388 - Follow official documentation, blogs, and forums. Participate
389
390 in community events, webinars, and online courses. Keep up with updates from Microsoft
    and other reputable sources in the developer community.
391
392 Certainly! Here's a concise overview of the basic concepts of C#:
393
394 ### 1. **Basic Syntax and Structure**
395 - **Program Structure**: C# programs are organized into classes and namespaces. The
    entry point of a C# application is the `Main` method.
396     ``csharp
397     using System;
398
399     namespace HelloWorld
400     {
401         class Program
402         {
403             static void Main(string[] args)
404             {
405                 Console.WriteLine("Hello, World!");
406             }
407         }
408     }
409     ...
410
411 ### 2. **Data Types**
412 - **Primitive Types**: C# provides built-in types like `int`, `float`, `double`, `char`,
    `bool`, and more.
413     ``csharp
414     int age = 25;
415     float height = 5.9f;

```

```

416     char initial = 'A';
417     bool isStudent = true;
418     ```
419
420 - Complex Types: Includes `string`, arrays, and custom types like classes and
    structs.
421
422 3. Variables and Constants
423 - Variables: Store data that can be modified.
424     ```csharp
425     int number = 10;
426     ```
427
428 - Constants: Values that cannot be changed once set, declared with `const`.
429     ```csharp
430     const double Pi = 3.14;
431     ```
432
433 4. Operators
434 - Arithmetic Operators: `+`, `-`, `*`, `/`, `%`
435 - Relational Operators: `==`, `!=`, `>`, `<`, `>=`, `<=`
436 - Logical Operators: `&&`, `||`, `!`
437
438 5. Control Flow
439 - Conditionals: Used to make decisions.
440     ```csharp
441     if (age > 18)
442     {
443         Console.WriteLine("Adult");
444     }
445     else
446     {
447         Console.WriteLine("Not an adult");
448     }
449     ```
450
451 - Loops: Repeatedly execute a block of code.
452     ```csharp
453     // For loop
454     for (int i = 0; i < 5; i++)
455     {
456         Console.WriteLine(i);
457     }
458
459     // While loop
460     int j = 0;
461     while (j < 5)
462     {
463         Console.WriteLine(j);
464         j++;
465     }
466     ```
467
468 6. Methods
469 - Definition: Blocks of code that perform a task. Methods can take parameters and
    return values.
470     ```csharp
471     int Add(int a, int b)
472     {
473         return a + b;
474     }
475     ```
476
477 - Calling a Method:
478     ```csharp
479     int result = Add(5, 3);
480     ```
481
482 7. Classes and Objects

```



```

483 - **Classes**: Define data and behavior.
484     ```csharp
485     class Person
486     {
487         public string Name { get; set; }
488         public int Age { get; set; }
489
490         public void Greet()
491         {
492             Console.WriteLine($"Hello, my name is {Name}.");
493         }
494     }
495     ```
496
497 - **Objects**: Instances of classes.
498     ```csharp
499     Person person = new Person();
500     person.Name = "Alice";
501     person.Age = 30;
502     person.Greet();
503     ```
504
505 ### 8. **Inheritance**
506 - **Base Class**: A class from which other classes derive.
507     ```csharp
508     class Animal
509     {
510         public void Eat() { Console.WriteLine("Eating..."); }
511     }
512     ```
513
514 - **Derived Class**: Inherits from a base class.
515     ```csharp
516     class Dog : Animal
517     {
518         public void Bark() { Console.WriteLine("Barking..."); }
519     }
520     ```
521
522 ### 9. **Encapsulation**
523 - **Access Modifiers**: Control visibility of class members. Common modifiers are
524   `public`, `private`, `protected`, and `internal`.
525     ```csharp
526     class Car
527     {
528         private string _engineNumber;
529
530         public string EngineNumber
531         {
532             get { return _engineNumber; }
533             set { _engineNumber = value; }
534         }
535     }
536     ```
537
538 ### 10. **Polymorphism**
539 - **Method Overriding**: Allows a derived class to provide a specific implementation of
540   a method that is already defined in its base class.
541     ```csharp
542     class BaseClass
543     {
544         public virtual void Display() { Console.WriteLine("Base Class"); }
545     }
546
547     class DerivedClass : BaseClass
548     {
549         public override void Display() { Console.WriteLine("Derived Class"); }
550     }
551     ```

```

```

550
551 ### 11. **Interfaces**
552 - **Definition**: Contracts that define methods and properties without providing
implementation. Classes implement interfaces to adhere to these contracts.
553     ``csharp
554     interface IAnimal
555     {
556         void MakeSound();
557     }
558
559     class Dog : IAnimal
560     {
561         public void MakeSound() { Console.WriteLine("Woof!"); }
562     }
563     ``
564
565 ### 12. **Exception Handling**
566 - **Try-Catch-Finally**: Used to handle runtime errors.
567     ``csharp
568     try
569     {
570         int result = 10 / 0;
571     }
572     catch (DivideByZeroException ex)
573     {
574         Console.WriteLine("Cannot divide by zero.");
575     }
576     finally
577     {
578         Console.WriteLine("Cleanup code.");
579     }
580     ``
581
582 ### 13. **Collections**
583 - **Arrays**: Fixed-size collections of elements.
584     ``csharp
585     int[] numbers = { 1, 2, 3, 4, 5 };
586     ``
587
588 - **List<T>**: Dynamic-size collection from the `System.Collections.Generic` namespace.
589     ``csharp
590     List<string> names = new List<string> { "Alice", "Bob", "Charlie" };
591     ``
592
593 ### 14. **Delegates and Events**
594 - **Delegates**: Type-safe method references.
595     ``csharp
596     public delegate void Notify();
597
598     public class Process
599     {
600         public event Notify OnCompleted;
601
602         public void StartProcess()
603         {
604             // Process logic
605             OnCompleted?.Invoke();
606         }
607     }
608     ``
609
610 - **Events**: Provide a way to notify subscribers about changes.
611
612 ### 15. **LINQ (Language Integrated Query)**
613 - **Purpose**: Query collections using a SQL-like syntax.
614     ``csharp
615     var numbers = new List<int> { 1, 2, 3, 4, 5 };
616     var evenNumbers = from n in numbers
617                       where n % 2 == 0

```

618  
619

```

select n;